

Relatório de Análise de Vulnerabilidades

Projeto: API Oficina Mecânica (MVP)
Stack: Python, FastAPI, SQLAlchemy, PostgreSQL
Data do relatório: 10/05/2026
Escopo: Código em `app/`, dependências instaladas no ambiente de análise (`pip freeze`), e revisão de riscos arquiteturais.

1. Objetivo e metodologia

1.1 Objetivo

Identificar e classificar riscos de segurança (exposição de credenciais, configuração insegura, padrões de código frágeis, dependências vulneráveis) e documentar ações mitigatórias.

1.2 Ferramentas e escaneamentos realizados

Ferramenta | Versão (referência) | Escopo | Propósito
Bandit | 1.9.4 | Análise estática (SAST) em `app/` | Padrões inseguros em Python (bind em `0.0.0.0`, `assert` em produção, falsos positivos de “senha”)
pip-audit | (PyPI) | Base de vulnerabilidades (OSV) nas dependências | CVEs em pacotes instalados via `pip`
Comandos executados (reproduzíveis):

```
# A partir da raiz do repositório, com venv ativado
bandit -r app -f txt
pip freeze | pip-audit --requirement /dev/stdin
```

Nota: A auditoria de dependências usa o **conjunto de pacotes** listado por `pip freeze` no momento da análise. Em produção, recomenda-se integrar `bandit` e `pip-audit` (ou equivalente) em pipeline de CI com ambiente limpo e versões fixadas.

2. Resultado do scan estático (Bandit)

Resumo automático (execução em 10/05/2026):

- Linhas de código analisadas (LOC):** 2.224 (pasta `app/`, conforme métrica do Bandit)
- Problemas por severidade (Bandit):** Alta: 0 · Média: 1 · Baixa: 3
- Total de ocorrências:** 4

2.1 Ocorrências detalhadas

ID | Severidade | Arquivo:linha | Tipo (plugin) | Descrição (ferramenta)
B106 | Baixa | `app/application/use\_cases/auth.py:18` | `hardcoded\_password\_funcarg` | “Possível senha hardcoded” no argumento — **falso positivo**: o valor é `token\_type="bearer"` (padrão OAuth2), não uma credencial.
B104 | Média | `app/infrastructure/config/settings.py:21` | `hardcoded\_bind\_all\_interfaces` | Valor padrão `api\_host: str = "0.0.0.0"` (escuta em todas as interfaces). Comum em contêineres; em hosts expostos diretamente exige compensação (firewall, bind em `127.0.0.1`, `reverse proxy`),
B101 | Baixa | `app/main.py:56` | `assert\_used` | Uso de `assert isinstance(e, DomainError)` em `*handler*` — em bytecode **otimizado** (`python -O`), `*asserts*` são removidos.
B101 | Baixa | `app/main.py:62` | `assert\_used` | Uso de `assert isinstance(e, RequestValidationError)` no mesmo contexto.

2.2 Análise (interpretação)

- B106 / `bearer`:** Não há vazamento de segredo. O rótulo do Bandit é enganoso neste caso. **Mitigação opcional:** comentário `# nsec B106` na linha, se a política de SAST exigir suprimir o falso positivo.
- B104 / `0.0.0.0`:** Risco operacional, não lógica de aplicação. Em **Docker** é padrão ouvir `0.0.0.0` dentro do contêiner. Em **servidor bare metal** com IP público, restringir `*bind*` ou isolar com `*firewall*`. **Recomendação documentada no README.**

- **B101 / `assert` em `handlers`:** Risco teórico: com `python -O``, o tipo deixaria de ser verificado. Em Uvicorn/Gunicorn, em geral o interpretador **não** roda com `-O``. **Mitigação:** trocar `assert`` por `if not isinstance(...):`` e resposta de erro explícita, para o `handler`` ser robusto em qualquer modo de compilação.

Conclusão (Bandit): Nenhuma **vulnerabilidade crítica** ou **alta** reportada. Os itens requerem triagem: um falso positivo, um aviso de implantação e melhoria de estilo/robustez.

3. Resultado do scan de dependências (pip-audit)

3.1 Auditoria com base em `pip freeze`

Comando: `pip freeze | pip-audit --requirement /dev/stdin`

Achado relevante (dependência transitiva): o ecossistema puxava **Mako** (usado por **Alembic** para `templates`` de migration). Em 10/05/2026, o `pip-audit`` apontou:

Pacote | Versão afetada | ID | Versões com correção

mako | versões anteriores a 1.3.12 | **CVE-2026-44307** | **$\geq 1.3.12$**

Descrição resumida (OSV): vulnerabilidade no processamento de `templates`` Mako; o `upstream`` corrigiu na série 1.3.12.

Mitigação aplicada no projeto: o `pyproject.toml`` fixa a restrição **`mako>=1.3.12``**, garantindo resolução mínima segura em instalações futuras. Após atualizar o ambiente (`pip install -e .`` ou `pip install 'mako>=1.3.12``), a reexecução de `pip-audit`` passou a reportar **nenhuma vulnerabilidade conhecida** no `freeze`` analisado.

3.2 Pacote local `oficina-api`

O projeto instala-se em modo editável (`-e .``). O `pip-audit`` pode exibir **Skip Reason: not found on PyPI** para esse identificador — **comportamento esperado**; a auditoria aplica-se aos pacotes remotos (FastAPI, SQLAlchemy, Alembic, Mako, etc.).

3.3 Ferramenta `pip` (ambiente)

Em alguns ambientes, vulnerabilidades podem aparecer no próprio **pip** instalado no `venv``, e não nas bibliotecas da aplicação. Manter `python -m pip install --upgrade pip`` e repetir `pip-audit`` após `upgrades`` do ambiente.

Conclusão (dependências): Foi identificado e **corrigido via versão mínima** o achado em **Mako**; com o `freeze`` atualizado, **não há CVEs conhecidos** nas dependências auditadas. Manutenção contínua e `pin`` de versões em produção continuam recomendadas.

4. Revisão complementar (não automática)

Esta seção consolida riscos comuns em APIs similares e o comportamento do código analisado.

Tema | Avaliação resumida

Injeção SQL | Uso de SQLAlchemy com parâmetros vinculados; não há construção de SQL por concatenação de `strings`` identificada na camada de aplicação. Risco: **baixo** com manutenção do padrão.

Autenticação | JWT (HS256); depender de `JWT_SECRET`` forte e de HTTPS em produção. Risco: **médio** se segredo vazado ou tráfego em claro.

Exposição de OS (público) | `GET /public/ordens-servico/{id}`` sem autenticação. Qualquer pessoa que saiba o **id** pode consultar. Risco: **médio** (enumerar IDs); mitigar com `token`` de acesso de uma vez, assinatura ou id não sequencial se o modelo de ameaças exigir.

CORS | Padrão permissivo se `CORS_ORIGINS==```. Em produção, restringir origens.

Credenciais em repositório | `.env`` no `.gitignore``. Garantir que `.env`` e segredos **nunca** sejam versionados.

Cabeçalhos de segurança | Aplicação FastAPI não adiciona por padrão todos os cabeçalhos (HSTS, CSP); podem ser configurados no **reverse proxy** (Nginx, Traefik).

5. Recomendações priorizadas

- **Curto prazo**
 - Manter `mako >= 1.3.12` (e repetir `pip-audit` após cada alteração de dependências).
 - Corrigir `handlers` em `app/main.py` (substituir `assert` por verificação explícita de tipo).
 - Garantir `pip` atualizado no `venv` e integrar `bandit` + `pip-audit` no CI.
- **Médio prazo**
 - Revisar endpoint público de OS (risco de enumeração).
 - Fixar versões de dependências (`lock file`) em `deploy`.
 - Documentar requisito de TLS e tamanho mínimo de `JWT_SECRET`.
- **Documentação / processo**
 - Reexecutar este relatório após alterações significativas no código ou nas dependências.
 - Manter registro da data e do `hash` de `commit` de cada análise.

6. Conclusão geral

A combinação de **análise estática (Bandit)** e **auditoria de dependências (pip-audit)** no estado atual do repositório **não aponta vulnerabilidades críticas** no código escaneado; no **pip-audit**, o único CVE listado no ambiente de referência (**Mako / CVE-2026-44307**) está **mitigado** pela restrição de versão mínima. Permanecem **pontos de atenção** (`bind` `0.0.0.0`, uso de `assert` em `runtime`, falso positivo em `bearer`) e **riscos de desenho** (consulta pública de OS, CORS/HTTPS), a tratar conforme o perfil de ameaça e o ambiente de produção.

Documento para suporte a auditoria e melhoria contínua.

Versão PDF: o ficheiro `docs/relatorio_analise_vulnerabilidades.pdf` pode ser regenerado com:

```
pip install fpdf2
python3 scripts/gerar_relatorio_vulnerabilidades_pdf.py
```

(No Linux/WSL, o script usa fontes DejaVu do sistema para caracteres acentuados. Alternativa: abrir o `.md` no editor e **Imprimir → Guardar como PDF.)**

Ferramentas de scan (repetir a análise):

```
bandit -r app -f txt
pip freeze | pip-audit --requirement /dev/stdin
```